متن ارائه‌ی آزمایش ۴ — توصیف رفتاری

حدود ۵ دقیقه | پوریا عرفانزاده، فاطیما تیمارچی، آیه صابری

0:00 • شروع

سلام. من پوریا عرفانزاده هستم، با فاطیما تیمارچی و آیه صابری. آزمایش چهارم ما توصیف رفتاری بود و چیزی که ساختیم یه استک سخت‌افزاری با عمق ۸ و پهنای ۴ بیته.

0:20 • صورت مسئله

استک همون ساختار LIFO آشناست: هرچی آخر گذاشتی، اوّل برمی‌داری. ورودی‌هاش کلاک، ریست، داده، و دو تا فرمان Push و Popه؛ خروجی‌هاش داده‌ی خروجی و دو تا فلگ Full و Empty.

نکته‌ی این آزمایش این بود که برعکس آزمایش سوم که فقط dataflow مجاز بود، اینجا فقط توصیف رفتاری مجاز بود — یعنی always و case و انتساب non-blocking. ما رفتار مطلوب رو می‌نویسیم و سنتزکننده خودش تصمیم می‌گیره چه سخت‌افزاری بسازه.

0:55 • دو تا نکته‌ی ریز ولی مهم

اوّل اینکه فلگ Empty تو دستور کار فعّال‌صفره. صریح نوشته Empty=0 یعنی استک خالیه. یعنی برخلاف انتظار، وقتی Empty بالاست یعنی استک خالی نیست. این رو تو کل کد و تست‌بنچ رعایت کردیم و شکل موج‌ها رو هم باید با همین قرارداد خوند.

دوم اینکه دستور کار چند تا حالت مرزی رو اصلاً تعریف نکرده بود و خودمون باید تصمیم می‌گرفتیم. Push روی استک پر رو نادیده می‌گیریم که داده‌ی معتبر بازنویسی نشه. Pop از استک خالی رو هم نادیده می‌گیریم و Data_Out مقدار قبلیش رو نگه می‌داره — صفر کردنش کار درستی نبود، چون با یه داده‌ی واقعیِ صفر قاطی می‌شد. و Push و

Pop همزمان: هیچ کاری نمی‌کنیم. این ساده‌ترین و قابل‌تست‌ترین انتخاب بود و ما رو از تعریف یه اولویّت دلبخواهی نجات داد.

1:45 • اشاره‌گر و پهنای آن

هسته‌ی طرح یه رجیستره به اسم Stack_Pointer که تعداد عناصر معتبر رو نگه می‌داره. چون از صفر می‌شماره، همین عدد همزمان آدرس اوّلین خانه‌ی خالی هم هست؛ پس Push می‌نویسه روی خودِ اشاره‌گر و بعد یکی زیاد می‌کنه، و Pop از یکی کمتر می‌خونه و بعد یکی کم می‌کنه. متقارن و ساده.

یه نکته‌ی ظریف تو پهنای اشاره‌گره. ممکنه فکر کنید برای عمق ۸، سه بیت کافیه؛ ولی کافی نیست. اشاره‌گر باید ۹ تا مقدار مختلف رو نشون بده، از صفر تا ۸. اگه سه بیت می‌گرفتیم، مقدار ۸ که دقیقاً همون شرط پر بودنه اصلاً نمایش‌پذیر نبود و شمارنده به‌جاش برمی‌گشت رو صفر. پس فرمولش لگاریتم دوی عمق *به‌علاوه‌ی یک* می‌شه، که برای ما چهار بیته.

هر دو فلگ هم فقط از روی همین اشاره‌گر و به‌صورت ترکیبی ساخته می‌شن. اینکه هر دو از یه منبع واحد بیان، امکان ناسازگاری بین وضعیّت واقعی استک و چیزی که گزارش می‌شه رو از بین می‌بره.

2:35 • تست‌بنچ

تست‌بنچ self-checking و هفت مرحله داره: ریست، Pop از خالی، پر کردن کامل، Push روی پر، برداشت کامل با بررسی ترتیب LIFO، دوباره Pop از خالی، و آخرش فرمان هم‌زمان.

یه نکته‌ی مهم تو مرحله‌ی آخر هست: اونجا علاوه بر خروجی‌ها، خودِ اشاره‌گر رو هم مستقیم چک می‌کنیم. دلیلش اینه که اگه فقط به خروجی‌ها نگاه می‌کردیم، یه مدار غلط

که هم Push و هم Pop رو انجام می‌ده هم از تست سربلند بیرون میومد —— چون اثر دو تا عمل روی اشاره‌گر همدیگه رو خنثی می‌کنه. سنجش مستقیم اشاره‌گر این راه فرار رو می‌بنده.

3:25 • پارامتری بودن

طرح با دو تا پارامتر DEPTH و WIDTH نوشته شده و هیچ‌جای کد به عددهای ۸ و ۴ وابستگی سخت نداره. برای اینکه این فقط ادّعا نمونه، همون کد و همون تست‌بنچ رو یه بار دیگه با عمق ۴ و پهنای ۸ بیت اجرا کردیم —— فقط با سوییچ g- موقع لود کردن شبیه‌ساز، بدون یه خط تغییر تو کد. هر هفت مرحله بدون خطا پاس شدن.

3:55 • سنتز

تو Quartus طرح ۴۰ تا رجیستر و ۱۴ تا پین داد. هر دو عدد رو می‌شه مستقیم از روی کد حساب کرد: ۸ خانه‌ی چهاربیتی حافظه، به‌علاوه‌ی چهار بیت اشاره‌گر، به‌علاوه‌ی چهار بیت Data_Out می‌شه ۴۰؛ و ۸ پین ورودی و ۶ پین خروجی می‌شه ۱۴. اینکه عدد گزارش دقیقاً با محاسبه‌ی دستی می‌خونه، یعنی سنتزکننده همون چیزی رو ساخته که ما در نظر داشتیم.

یه نکته‌ی جالب هم بود: تو RTL viewer آرایه‌ی حافظه به‌صورت SYNC_RAM استنتاج شده، ولی گزارش منابع می‌گه صفر بیت حافظه مصرف شده. دلیلش اینه که کلّ حافظه‌ی استک فقط ۳۲ بیته و ابزار ترجیح داده به‌جای اشغال یه بلاک RAM اختصاصی، با همون رجیسترهای معمولی بسازتش —— همون ۴۰ رجیستری که تو گزارش می‌بینیم.

آخرش طرح رو روی یه قطعه‌ی مشخّص از Cyclone II بردیم و هر ۱۴ پین رو با استاندارد 3.3-V LVTTL نسبت دادیم. مقایسه‌ی دو قطعه جالب بود: تعداد رجیسترها دقیقاً ثابت موند، چون رجیستر از ساختار طرح میاد نه از فناوری قطعه؛ ولی Fmax از ۲۵۲ اومد رو

۲۱۵ مگاهرتز، چون Cyclone II فناوری قدیمی‌تریه. تحلیل زمانی هم با قید ۵۰ مگاهرتز slack مثبت داد.

4:45 • جمع‌بندی

درس اصلی این آزمایش فاصله‌ی بین «بیان رفتار» و «بیان ساختار» بود — کلّ منطق استک تو چند ده خط و بدون هیچ اشاره‌ای به گیت یا فلیپ‌فلاپ نوشته شد و سنتزکننده خودش حافظه و شمارنده و مقایسه‌کننده‌ها رو استنتاج کرد. در کنارش، دقّت رو همون چیزهایی که دستور کار تعریف نکرده بود — قطبیّت Empty، رفتار در برابر overflow و underflow، و معنای فرمان هم‌زمان — تعیین‌کننده‌ی درستی طرح بود. چون آزمایشگاه مجازی بود روی برد فیزیکی پروگرام نکردیم. ممنون، اگه سؤالی هست در خدمتم.